Now that the input and output are clear, it's time to define the function. Open the `DispatchingFunctions` class and add a `label()` method to implement the order labeling as a function. Since we want it to be reactive, both input and output are wrapped in a `Flux` publisher.

Listing 10.8 Implementing the "label" action as a function

```
package com.polarbookshop.dispatcherservice;

import java.util.function.Function;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import reactor.core.publisher.Flux;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DispatchingFunctions {
  private static final Logger log =
    LoggerFactory.getLogger(DispatchingFunctions.class);

  ...

  @Bean                                                              ❶
  public Function<Flux<Long>, Flux<OrderDispatchedMessage>> label() {
    return orderFlux -> orderFlux.map(orderId -> {                   ❷
      log.info("The order with id {} is labeled.", orderId);
      return new OrderDispatchedMessage(orderId);                    ❸
    });
  }
}
```

❶ Function implementing the order-labeling business logic

❷ It takes an order identifier (Long) as input.

❸ Returns an OrderDispatchedMessage as output

We have just implemented both functions, so let's see how we can combine and use them.

10.3.2 Composing and integrating functions: REST, serverless, data streams